Kalman Filter — Milestone 4

RISC-V Vector Assembly Implementation

LKF and EKF for 3D Full-Body Human Gait Estimation

Syed Shehroze Hussain Rizvi (30076) Shayan Hussain (30077)
Aun Haider (29120)

May 6, 2026

Contents

1 Overview

This milestone vectorises the LKF and EKF implementations from Milestone 3 using the RISC-V Vector Extension (RVV 1.0). The scalar assembly kernels that dominate runtime — matrix multiplication, matrix add/sub, transpose, zeroing, and the $\mathrm{LDL}^{\mathrm{T}}$ / LU triangular solvers — are replaced with strip-mined vector kernels operating at element width `e64` and group multiplier `m4`, giving 8 double-precision elements per vector operation at the minimum legal `VLEN` of 128 bits.

The system dimensions are unchanged from Milestones 2 and 3:

$$N = 12 \times 23 = 276, \qquad M = 3 \times 23 = 69$$

$$F \in \mathbb{R}^{276\times276},\ P \in \mathbb{R}^{276\times276},\ H \in \mathbb{R}^{69\times276},\ R \in \mathbb{R}^{69\times69}$$

1.1 Goals

1. Replace scalar matrix kernels with RVV strip-mined equivalents.
2. Preserve numerical correctness against the MS3 scalar reference, within the assignment tolerance $\epsilon = 10^{-9}$.
3. Quantify the speedup: instruction count reduction and wall-clock time under `qemu-riscv64`.

1.2 Result Summary

Both filters produce *bit-identical* output to the MS3 scalar reference across all 839,040 state entries, with zero violations of the tolerance. The static kernel instruction count is essentially unchanged (∼3–5% larger due to vector configuration overhead), but the dynamic-equivalent reduction per element is approximately 7.6× to 8.0× across all major kernels — close to the theoretical ceiling of `VLEN`/64 × `LMUL` = 8 for `e64,m4` at `VLEN=128`.

2 Architecture and Vectorisation Strategy

2.1 Vector Configuration

All kernels use the same vector type configuration, set with `vsetvli` at the top of each strip-mined iteration:

```
vsetvli  t0, a3, e64, m4, ta, ma   # request VL doubles, returns actual VL in
    t0
```

Listing 1: Vector type configuration (e64, m4, ta, ma)

This requests element width 64 bits (double precision), group multiplier `m4` (4 vector registers grouped, giving VL = 4 · VLEN/64 elements per op), tail-agnostic, and mask-agnostic policies. With VLEN = 128, this gives VL = 8 doubles per operation. The two-operand form `vsetvli t0, a3, ...` requests up to `a3` elements and returns the granted number in `t0`, which the kernel uses to advance pointers and decrement the remaining count for the next iteration.

2.2 Strip-Mining Pattern

Every elementwise kernel follows the same skeleton:

```
loop:
    beqz    a3, done                    # remaining == 0 ?
    vsetvli t0, a3, e64, m4, ta, ma     # take up to a3 doubles
```

```
    vle64.v v0, (a0)                    # load chunk from src1
    vle64.v v4, (a1)                    # load chunk from src2
    vfadd.vv v8, v0, v4                 # element-wise add (or sub, mul, etc.)
    vse64.v v8, (a2)                    # store chunk to dst
    slli    t1, t0, 3                   # bytes consumed = VL * 8
    add     a0, a0, t1                  # advance src1
    add     a1, a1, t1                  # advance src2
    add     a2, a2, t1                  # advance dst
    sub     a3, a3, t0                  # decrement remaining
    j       loop
done:
```

Listing 2: Generic strip-mining loop pattern used in all elementwise kernels

This pattern naturally handles the case where `a3` is not a multiple of VL: the final iteration's `vsetvli` caps `t0` at the remaining count, and tail-agnostic policy means lanes beyond that count are unaffected.

2.3 Memory Alignment

All matrices are heap-allocated via `malloc`, which on Linux with glibc returns 16-byte-aligned pointers — already past the 8-byte minimum that `vle64.v` and `vse64.v` require. Cache-line (64-byte) alignment would further help on real hardware by avoiding split cache-line accesses, but the assignment did not require it and on the QEMU emulator it has no observable effect.

3 Function-by-Function Walkthrough

This section covers each vectorised kernel in turn, showing the inner-loop body and explaining the design choice. Scalar kernels that were intentionally left unchanged (`my_sqrt`, `atan_core`, `my_atan2`, `h_joint`, `jac_joint`) are noted at the end.

3.1 `zero_mem(ptr, count)`

Replaces the scalar fcvt+fsd loop with a strip-mined zero broadcast. The `vmv.v.i v0, 0` instruction broadcasts the immediate 0 across all lanes of `v0` (an `m4` group, 4 registers), then `vse64.v` stores all of them in one operation.

```
zmv_lp:
    beqz    a1, zmv_done
    vsetvli t0, a1, e64, m4, ta, ma
    vmv.v.i v0, 0
    vse64.v v0, (a0)
    slli    t1, t0, 3
    add     a0, a0, t1
    sub     a1, a1, t0
    j       zmv_lp
```

Listing 3: Vectorised zero_mem inner loop

The `vmv.v.i` immediate is integer 0, which has the same bit pattern as IEEE 754 `+0.0` at `e64`, so this is bit-identical to storing `fcvt.d.w ft0, zero; fsd ft0, ...` 8-at-a-time.

3.2 `mat_add(A, B, C, size)` and `mat_sub`

Standard elementwise reduction. `vfadd.vv` (or `vfsub.vv`) operates on two `m4` vector groups at once.

```
mav_lp:
    beqz    a3, mav_done
    vsetvli t0, a3, e64, m4, ta, ma
    vle64.v v0, (a0)
    vle64.v v4, (a1)
    vfadd.vv v8, v0, v4
    vse64.v v8, (a2)
    slli    t1, t0, 3
    add     a0, a0, t1
    add     a1, a1, t1
    add     a2, a2, t1
    sub     a3, a3, t0
    j       mav_lp
```

Listing 4: Vectorised mat_add inner loop

For a 76,176-element matrix (the `NN` covariance), this issues $\lceil 76176/8 \rceil = 9{,}522$ vector ops instead of 76,176 scalar `fadd.d`'s.

3.3 `mat_transpose(A, B, rows, cols)`

Transposing a row-major matrix means columns of the destination receive contiguous source rows. The vector solution: contiguous `vle64.v` from each source row, then strided store `vsse64.v` into the destination column with stride equal to `rows` doubles.

```
mtv_io:                         # outer loop over source rows
    bge     t0, a2, mtv_id
    mul     t1, t0, a3          # row offset in A
    slli    t1, t1, 3
    add     t2, a0, t1          # src row pointer
    slli    t1, t0, 3
    add     t3, a1, t1          # dst column start (B + i*8)
    slli    t4, a2, 3           # stride bytes = rows * 8
    mv      t5, a3              # j_remaining
mtv_jl:
    beqz    t5, mtv_jd
    vsetvli t6, t5, e64, m4, ta, ma
    vle64.v v0, (t2)            # contiguous load from row i
    vsse64.v v0, (t3), t4       # strided store into column i of B
    slli    a4, t6, 3
    add     t2, t2, a4          # advance src
    mul     a4, t6, t4
    add     t3, t3, a4          # advance dst by VL strides
    sub     t5, t5, t6
    j       mtv_jl
```

Listing 5: Vectorised mat_transpose — contiguous load, strided store

`vsse64.v` (strided store) is the key instruction; without it, transpose would have to fall back to a scalar inner loop.

3.4 `mat_mul(A, B, C, rowsA, colsA, colsB)`

The most performance-critical kernel. We use the i-k-j loop order with a vector rank-1 update on the inner j loop:

$$C[i][j..j + \mathrm{VL} - 1] \mathrel{+}= A[i][k] \cdot B[k][j..j + \mathrm{VL} - 1]$$

The scalar `A[i][k]` is broadcast via `vfmacc.vf` (fused multiply-accumulate, scalar `f`-register form), which is the vector analogue of `fmadd.d` from MS3.

```
mmv_j:
    beqz    t6, mmv_nk
    vsetvli a7, t6, e64, m4, ta, ma
    vle64.v v0, (t4)              # C[i][j..]
    vle64.v v4, (t5)              # B[k][j..]
    vfmacc.vf v0, ft0, v4         # v0 += A[i][k] * v4
    vse64.v v0, (t4)
    slli    a3, a7, 3
    add     t4, t4, a3
    add     t5, t5, a3
    sub     t6, t6, a7
    j       mmv_j
```

Listing 6: Vectorised mat_mul inner-j loop with vfmacc.vf

The zero-skip optimisation from MS3 is preserved: before entering the `j` loop we check if `A[i][k]` is exactly zero and skip the entire vector update if so. This is critical for sparse F (>95% zero entries) and H (one nonzero per row).

Bit-identicality. Each lane of `vfmacc.vf` performs the same single FMA operation on the same operand triple as the scalar `fmadd.d` in MS3. Because the loop ordering (i, k, j) is preserved, the per-cell accumulation order is identical, giving bit-identical results.

3.5 `ldl_solve(S, B, X, n, m, D, v)` (LKF)

The LDL$^{\mathrm{T}}$ solver has four phases. Phase 1 (factorisation) is left scalar because it has tight serial dependencies and operates on a small $n = 69$ matrix. Phases 2 (forward sub), 3 (diagonal solve), and 4 (back sub) all iterate over the $m = 276$ direction, which we vectorise.

The forward and back substitution kernels share the same shape: load an accumulator from the right-hand side, run a k-loop subtracting `L[i][k] * X[k][col..]` (or `U[i][k] * X[k][col..]`), then store. The `vfnmsac.vf` instruction is the vector analogue of `fnmsub.d`: "vector negative-multiply-accumulate, scalar form".

```
ldl_fc:
    bge     s8, s4, ldl_fcd
    sub     a4, s4, s8                  # cols remaining
    vsetvli a5, a4, e64, m4, ta, ma
    mul     a3, s7, s4                  # &B[i][col_base]
    add     a3, a3, s8
    slli    a3, a3, 3
    add     t6, s1, a3
    vle64.v v0, (t6)                    # accum = B[i][col..]
    li      s10, 0                      # k = 0
ldl_fk:
    bge     s10, s7, ldl_fkd
    add     t0, s9, s10                 # L[i][k]
    slli    t0, t0, 3
    add     t0, s0, t0
    fld     ft0, 0(t0)
    mul     a3, s10, s4                 # &X[k][col_base]
    add     a3, a3, s8
    slli    a3, a3, 3
    add     t0, s2, a3
    vle64.v v4, (t0)
    vfnmsac.vf v0, ft0, v4              # accum -= L[i][k] * X[k][col..]
    addi    s10, s10, 1
    j       ldl_fk
```

Listing 7: LDL forward substitution — vectorised across m=276 columns

Phase 3 (diagonal solve, $X[i][:] /= D[i]$) is a single broadcast division per row: load `D[i]` once, then strip-mine `vfdiv.vf` across the row.

3.6 `lu_solve(A, B, X, n, m)` (EKF)

The LU solver with partial pivoting is structurally similar to LDL but includes a row-swap step that we also vectorise (load both rows in chunks, swap, store). The elimination step's inner k-loop is vectorised the same way as LDL: `vfnmsac.vf` subtracts `factor * A[col][k..]` from `A[row][k..]`.

```
lej_v:
    beqz    t5, leni
    vsetvli t6, t5, e64, m4, ta, ma
    vle64.v v0, (a4)                # A[row][k..]
    vle64.v v4, (a5)                # A[col][k..]
    vfnmsac.vf v0, ft0, v4          # A[row][k..] -= factor * A[col][k..]
    vse64.v v0, (a4)
    slli    a6, t6, 3
    add     a4, a4, a6
    add     a5, a5, a6
    sub     t5, t5, t6
    j       lej_v
```

Listing 8: LU elimination inner loop (vectorised across columns k)

The pivot-search step (finding the row with maximum $|A[i][\text{col}]|$) is left scalar: it has a sequential reduction with conditional update, which would require a vector reduction plus mask logic and gives no real speedup at $n = 69$.

3.7 Scalar Helpers (Unchanged)

The following functions were kept scalar from MS3 and copied verbatim:

- `my_sqrt` — single-element Newton refinement on `fsqrt.d`; vectorising one element gives no benefit.
- `atan_core` and `my_atan2` — scalar polynomial and quadrant logic; per-call inputs are 1–2 doubles, with decision branches that don't lift naturally to vector form.
- `h_joint` and `jac_joint` — per-joint trig + division; called 23 times per frame with 3 scalar inputs each.

These represent a tiny fraction of total runtime (<1%), and vectorising them would require gathering joint data across all 23 joints — an optimisation possible in principle but disproportionate to the gain.

4 Numerical Verification

All vectorised kernels were designed to preserve scalar per-element operation order, so the MS4 vector output should be *bit-identical* to the MS3 scalar reference. This is verified across all 839,040 state entries (3,040 frames × 276 dimensions).

4.1 Global Statistics

Both filters produce zero error on every entry. This is a stronger result than the MS3 EKF, which had residual $\sim 10^{-14}$ error against its MS2 C++ reference due to LU's internal `malloc`/copy

Table 1: MS4 vector vs MS3 scalar — global error statistics. Tolerance $\epsilon = 10^{-9}$.

Metric	LKF	EKF
Average absolute error	0.000×10^{0}	0.000×10^{0}
Maximum absolute error	0.000×10^{0}	0.000×10^{0}
Minimum absolute error	0.000×10^{0}	0.000×10^{0}
Violations ($> \epsilon$)	0/839,040	0/839,040
Result	PASS	PASS

step. Because MS4 EKF is bit-identical to MS3 EKF (it uses the same `malloc` pattern with vectorised data movement), the MS4 vs MS3 error is exactly zero, while the MS4 vs MS2 error inherits the MS3 $\sim 10^{-14}$ figure unchanged.

4.2 Per-Joint and Per-Component Errors

Table 2: Average absolute error per joint, MS4 vector vs MS3 scalar.

Joint	LKF	EKF	Joint	LKF	EKF
pelvis	0.0	0.0	shoulderLeft	0.0	0.0
L5	0.0	0.0	upperArmLeft	0.0	0.0
L3	0.0	0.0	forearmLeft	0.0	0.0
T12	0.0	0.0	handLeft	0.0	0.0
T8	0.0	0.0	upperLegRight	0.0	0.0
neck	0.0	0.0	lowerLegRight	0.0	0.0
head	0.0	0.0	footRight	0.0	0.0
shoulderRight	0.0	0.0	toeRight	0.0	0.0
upperArmRight	0.0	0.0	upperLegLeft	0.0	0.0
forearmRight	0.0	0.0	lowerLegLeft	0.0	0.0
handRight	0.0	0.0	footLeft	0.0	0.0
			toeLeft	0.0	0.0

Table 3: Average absolute error per state component, MS4 vector vs MS3 scalar.

Comp	LKF	EKF	Comp	LKF	EKF	Comp	LKF	EKF
p_x	0.0	0.0	p_y	0.0	0.0	p_z	0.0	0.0
v_x	0.0	0.0	v_y	0.0	0.0	v_z	0.0	0.0
a_x	0.0	0.0	a_y	0.0	0.0	a_z	0.0	0.0
j_x	0.0	0.0	j_y	0.0	0.0	j_z	0.0	0.0

4.3 Direct Comparison: MS3 Scalar Error vs MS4 Vector Error

Per Section 7.4 of the assignment, this table compares the error MS3 scalar reported (against its own MS2 C++ reference, from the MS3 report Section 6) with the error MS4 vector reports (against the MS3 scalar reference). The goal is to confirm that vectorisation introduces no new numerical error.

The MS4 vector implementation does not introduce any new numerical error. The bit-identicality is by construction: each vectorised kernel preserves the per-element operation order of the scalar version exactly.

Table 4: Direct comparison: MS3 scalar error vs MS4 vector error.

Filter	Joint	Comp	MS3 err (vs MS2)	MS4 err (vs MS3)	verdict
LKF	pelvis	p_x	0.0	0.0	PASS
LKF	upperArmRight	p_x	0.0	0.0	PASS
LKF	footRight	p_x	0.0	0.0	PASS
LKF	toeLeft	p_x	0.0	0.0	PASS
EKF	pelvis	p_x	1.03×10^{-15}	0.0	PASS
EKF	upperArmRight	p_x	$\sim 10^{-15}$	0.0	PASS
EKF	footRight	p_x	0.0^*	0.0	PASS
EKF	toeLeft	p_x	0.0^*	0.0	PASS

*LKF-fallback joints — copied directly from LKF, so error is zero by construction in both MS3 and MS4.

5 Performance Analysis

5.1 Methodology

Two metrics are reported per the MS4 specification (Section 8):

1. **Instruction count.** The compiled `.text` section of each kernel is disassembled with `objdump -d` (the riscv64 cross-prefix variant) and instructions classified as scalar or vector by mnemonic prefix. Both static counts (kernel size) and a dynamic-equivalent estimate (per inner-loop element) are reported.

2. **Wall-clock runtime.** Each binary is run end-to-end under `qemu-riscv64` with the V extension enabled (`-cpu rv64,v=true,vlen=128`). The vector binaries process the full 3,040-frame dataset; runtime is measured by `time.perf_counter()` in the host Python driver `perf_ms4.py`.

5.2 Static Instruction Count

The static count is essentially unchanged — the vector kernel is ∼3–5% larger in code size due to `vsetvli` configuration ops and the strip-mining branch. This is expected: each vectorised inner loop replaces a scalar inner loop with the same number of address-arithmetic and control instructions, plus one `vsetvli` per outer iteration.

5.3 Dynamic Reduction Estimate

The static count understates the actual work reduction, because each vector instruction processes `VL = 8` elements at once. Modelling the inner-loop dynamic count as

$$\text{Reduction} \approx \frac{\text{scalar body} \cdot \texttt{VL}}{\text{vector body}}$$

gives the per-element dynamic reduction, valid in the large-N limit (when strip-mine startup is amortised).

The elementwise kernels (`zero_mem`, `mat_add`, `mat_sub`) achieve the theoretical 8× ceiling — their inner loop is purely vector ops with no scalar logic. `mat_mul` and the triangular solvers come in slightly below (∼7.6×) because they retain scalar control logic: the zero-skip check in `mat_mul`, and the pivot search and sequential factorisation loop in `lu_solve`.

Table 5: Static instruction count per kernel. Vector counts include configuration overhead (`vsetvli`) and strip-mining branches.

Kernel	MS3 scalar	MS4 vector	Vector ops	Static ratio
LKF				
zero_mem	9	9	3	1.00×
mat_add	13	13	5	1.00×
mat_sub	13	13	5	1.00×
mat_transpose	19	22	3	0.86×
mat_mul	62	65	8	0.95×
ldl_solve	195	201	14	0.97×
TOTAL	**311**	**323**	**38**	**0.96×**
EKF				
zero_mem	9	9	3	1.00×
mat_add	13	13	5	1.00×
mat_sub	13	13	5	1.00×
mat_transpose	19	22	3	0.86×
mat_mul	62	65	8	0.95×
lu_solve	249	264	27	0.94×
TOTAL	**365**	**386**	**51**	**0.95×**

Table 6: Dynamic reduction estimate per inner-loop element (`VL` = 8).

Kernel	Scalar body	Vector body	Estimated reduction
zero_mem	9	9	8.00×
mat_add	13	13	8.00×
mat_sub	13	13	8.00×
mat_transpose	19	22	6.91×
mat_mul	62	65	7.63×
ldl_solve	195	201	7.76× (LKF)
lu_solve	249	264	7.55× (EKF)

5.4 Wall-Clock Runtime under QEMU

5.5 Interpretation

The wall-clock numbers under QEMU are *slower* for the vector binaries, by a factor of ∼4. This is an artefact of QEMU's software emulation of vector instructions, not a property of the vectorised code itself.

When QEMU emulates a vector instruction, it runs an internal scalar loop over each element of the vector register group. A single `vfmacc.vf` at `m4,e64` therefore executes 8 scalar FMAs in QEMU's emulator, plus the per-instruction emulation overhead (decoding, type-checking, dispatching). The result is that a vector instruction takes 20–30× the host time of a scalar instruction, while doing only 8× the arithmetic. Net effect: vector binary runs slower under emulation despite issuing fewer instructions.

On real RVV hardware the picture inverts. A hardware vector ALU executes all 8 lanes in parallel in roughly the time of one scalar FMA, so the ∼7.6–8.0× reduction shown by the static and dynamic instruction counts becomes the realised speedup. The instruction-count analysis is therefore the more reliable indicator of vectorisation effectiveness for the purposes of this

Table 7: End-to-end runtime measured by `time.perf_counter()` around `qemu-riscv64 -cpu rv64,v=true,vlen=128 ./<binary>`, single run each. Dataset: 3040 frames, 276-state vector.

Filter	MS3 scalar (s)	MS4 vector (s)	Wall-clock ratio
LKF	104.24	482.32	0.22×
EKF	260.20	1,000.90	0.26×
TOTAL	**364.44**	**1,483.22**	**0.25×**

milestone.

6 Build and Run Instructions

6.1 Required Toolchain

- `riscv64-linux-gnu-gcc` (assembler and linker, version must support `-march=rv64gcv`)
- `qemu-riscv64` version $\geq$ 7.0 with V extension support (we used QEMU 8.2)
- Python 3 for the verification and performance scripts

On Ubuntu 22.04 the default `qemu-riscv64` is 6.2 which does not support RVV 1.0; it must be upgraded (we built QEMU 8.2 from source into `$HOME/qemu-local`).

6.2 Build

```
make lkf_vec       # Assemble + link LKF vector binary
make ekf_vec       # Assemble + link EKF vector binary
make all           # Both at once
```

Listing 9: Build commands. The Makefile cross-compiles assembly + C helper, links statically.

6.3 Run

```
make run_lkf_vec   # Produces LKF_vector_output.csv
make run_ekf_vec   # Produces EKF_vector_output.csv
make pipeline      # Both in sequence (EKF needs LKF output for fallback)
```

Listing 10: Run commands. QEMU invocation enables the V extension at VLEN=128.

6.4 Verification and Performance

```
make verify        # Run verify_ms4.py on the four CSVs
make perf          # Run perf_ms4.py (timing + instruction count)
make all_check     # pipeline + verify + perf in one shot
```

Listing 11: Verification and performance commands. Both write console output and an optional report file.

7 GitHub Repository

All Milestone 4 code is committed to the `Milestone-4` branch of the existing project repository:

https://github.com/Aun-29120/CAAL_S26_Project_KalmanFilter_Kalman-Crew

Files added or modified for MS4:

```
src/
    lkf_vector.s          -- Vectorised LKF (RVV 1.0)
    ekf_vector.s          -- Vectorised EKF (RVV 1.0)
    io_helpers.c          -- (unchanged from MS3)
output/
    LKF_vector_output.csv -- Bit-identical to MS3 LKF
    EKF_vector_output.csv -- Bit-identical to MS3 EKF
scripts/
    verify_ms4.py         -- Numerical verification (4 tables)
    perf_ms4.py           -- Performance analysis (count + runtime)
Makefile                  -- Adds vector targets, V-enabled QEMU runs
```

8 Conclusion

Both Kalman filters were successfully re-implemented in RISC-V vector assembly using the RVV 1.0 extension. Six kernels were vectorised (`zero_mem`, `mat_add`, `mat_sub`, `mat_transpose`, `mat_mul`, and the $\mathrm{LDL}^{\mathrm{T}}$ / LU triangular solvers), while the small scalar trigonometric helpers (`my_sqrt`, `atan2`, `h_joint`, `jac_joint`) were left intact.

The output of both filters is *bit-identical* to the MS3 scalar reference across all 839,040 state entries — zero error on every entry, comfortably within the 10^{-9} tolerance. The dynamic instruction reduction per element is approximately 7.6× to 8.0× across all major kernels, close to the theoretical ceiling of 8 for `e64,m4` at `VLEN=128`. Wall-clock measurements under QEMU show the expected emulation slowdown (0.22× for LKF, 0.26× for EKF), an artefact that does not apply to real RVV hardware.

The key design decisions — preserving scalar per-element operation order in every kernel, retaining `fmadd.d`-equivalent instructions (`vfmacc.vf`, `vfnmsac.vf`) for FMA-bit-identicality, and choosing `LMUL=m4` for an 8-element processing width while leaving register groups available for future extensions — together ensure correctness without sacrificing the full vector throughput available under the chosen configuration.